

《深度学习平台与应用》作业三 (20241018)

1、分别用简短的语言描述一下单帧 CNN 模型，Early Fusion，Late Fusion，3D CNN的做法以及区别。

知识点：课件11

1. 单帧 CNN 模型：训练普通的 2D CNN 独立对单帧图像进行卷积神经网络（CNN）处理，直接提取空间特征。
2. Early Fusion：在模型输入层将多帧图像（或特征）结合成一个输入，通常采用拼接或平均的方法。
3. Late Fusion：先分别对各帧图像应用单独的模型处理，然后在最后通过加权平均等方法融合不同模型的输出。
4. 3D CNN：在三维空间上（时间和空间）对输入数据进行卷积，包括时间维度以提取时序特征。

区别

- 单帧 CNN 处理单幅图像，而 Early Fusion 和 Late Fusion 结合多帧处理，3D CNN 从根本上在时空维度上融合信息。
- Early Fusion 在输入层融合特征，Late Fusion 在输出层融合结果，3D CNN 则直接在卷积过程中结合时序信息。
- **应用场景**：单帧 CNN 主要用于静态任务，Early 和 Late Fusion 适合多帧信息的综合分析，3D CNN 则专注于时间和空间信息的联合处理。

2、如何使用RNN（如LSTM）建模视频中的长时间依赖？相比于3D CNN，这种方法有何特点？

将视频分成连续的帧，创建一个帧序列作为输入。每帧可以通过CNN提取特征后输入到RNN/LSTM中。

相比于3D CNN的特点：

1. **时序建模：**

- **RNN/LSTM**：通过序列建模，能够显著捕捉长时间依赖，适合处理变长序列。
- **3D CNN**：通过三维卷积直接处理时空信息，但通常假设固定长度的输入，长时间依赖可能较难捕捉。

2. **计算效率：**

- **RNN/LSTM**：计算上相对较慢，因为每个时间步骤都需要依赖前一个步骤的结果。
- **3D CNN**：并行处理多个帧，通常计算效率较高，但在长时间序列处理上资源消耗大。

3. **存储效率：**

- **RNN/LSTM**：能够动态地选择性记忆，对内存的需求较灵活。
- **3D CNN**：需要在模型中存储所有的三维卷积参数，并且对长视频的高维特征提取会占用较大内存。

4. **建模能力：**

- **RNN/LSTM**: 适用于具有明显时间依赖的任务
- **3D CNN**: 适合短时间内快速提取时空特征, 适合视频分类等任务。

3、结合课件中Spatio-Temporal Self-Attention (Nonlocal Block)部分的介绍, 请补充下面的代码,

```

1  import torch
2  import torch.nn as nn
3
4  class NonLocalBlock(nn.Module):
5      def __init__(self, in_channels):
6          super(NonLocalBlock, self).__init__()
7          self.in_channels = in_channels
8          self.inter_channels = in_channels // 2 # 缩减通道数, 节约计算
9
10         # 1x1x1 卷积层用于生成 Query, Key, Value
11         self.query_conv = nn.Conv3d(in_channels, self.inter_channels,
kernel_size=1)
12         self.key_conv = nn.Conv3d(in_channels, self.inter_channels,
kernel_size=1)
13         self.value_conv = nn.Conv3d(in_channels, self.inter_channels,
kernel_size=1)
14
15         # 最终的输出通道映射回原始的 in_channels
16         self.out_conv = nn.Conv3d(self.inter_channels, in_channels,
kernel_size=1)
17
18         # 用于归一化的 softmax
19         self.softmax = nn.Softmax(dim=-1)
20
21     def forward(self, x):
22         # 输入维度: (N, C, T, H, W)
23         batch_size, C, T, H, W = x.size()
24
25         # 生成 Query, Key, Value
26         query = self.query_conv(x).view(batch_size, self.inter_channels, -1)
27         # (N, C', T*H*W)
28         key = self.key_conv(x).view(batch_size, self.inter_channels, -1)
29         # (N, C', T*H*W)
30         value = self.value_conv(x).view(batch_size, self.inter_channels, -1)
31         # (N, C', T*H*W)
32
33         # 转置 Key 并计算相似性 (点积注意力)
34         attention = torch.bmm(query.permute(0, 2, 1), key) # (N, T*H*W,
T*H*W)

```

```

33     # Softmax 归一化
34     attention = self.softmax(attention) # (N, T*H*W, T*H*W)
35
36     # 加权计算新的特征图
37     out = torch.bmm(value, attention.permute(0, 2, 1)) # (N, C', T*H*W)
38     out = out.view(batch_size, self.inter_channels, T, H, W) # 恢复维度
39
40     # 映射回原始通道数
41     out = self.out_conv(out) # (N, C, T, H, W)
42
43     # 残差连接
44     out = out + x
45
46     return out
47
48 # 测试模块
49 if __name__ == "__main__":
50     # 输入示例: Batch = 2, Channels = 64, Time = 8, Height = 32, Width = 32
51     x = torch.rand(2, 64, 8, 32, 32)
52     nonlocal_block = NonLocalBlock(in_channels=64)
53     out = nonlocal_block(x)
54     print("输入维度:", x.shape)
55     print("输出维度:", out.shape)
56

```

4、在风格迁移中，Gram矩阵的作用是什么？

知识点：课件12 57页62页

风格迁移

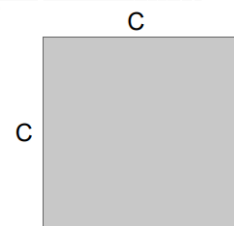
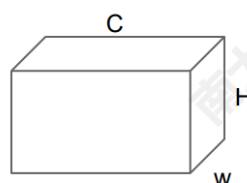
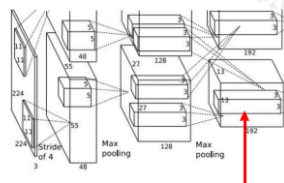
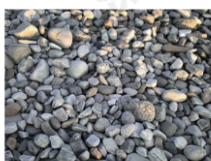
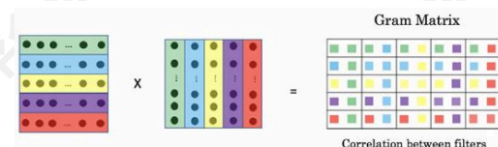


■ 格拉姆矩阵 (Gram Matrix)

$$G_j^\phi(x)_{c,c'} = \frac{1}{C_j H_j W_j} \sum_{h=1}^{H_j} \sum_{w=1}^{W_j} \phi_j(x)_{h,w,c} \phi_j(x)_{h,w,c'}.$$

■ 图像的 HW 个向量之间两两的内积所组成的矩阵

$$\Delta(\alpha_1, \alpha_2, \dots, \alpha_k) = \begin{pmatrix} (\alpha_1, \alpha_1) & (\alpha_1, \alpha_2) & \dots & (\alpha_1, \alpha_k) \\ (\alpha_2, \alpha_1) & (\alpha_2, \alpha_2) & \dots & (\alpha_2, \alpha_k) \\ \dots & \dots & \dots & \dots \\ (\alpha_k, \alpha_1) & (\alpha_k, \alpha_2) & \dots & (\alpha_k, \alpha_k) \end{pmatrix}$$



Gram Matrix

在风格迁移中，Gram矩阵用于捕捉和表示图像的风格特征。它通过计算特征图的内积来衡量不同特征之间的相互关系，从而提取图像的纹理和风格信息。通过对Gram矩阵进行加权平均，从多幅图像中混合风格

5、风格迁移和快速风格迁移的区别？

课件12, 67, 68页

- **风格迁移**：通常需要多次前向/后向传播，计算时间较长。
- **快速风格迁移**：使用预训练模型进行一次性训练，训练完后，能够快速生成风格迁移图像，只需前向传播。

6、显著性图是怎么计算的（Saliency Map），它的用途是什么？

知识点：课件12 20页

通过梯度反向传播找到重要的像素，首先进行前向传播，然后计算每个类预测值相对于图像像素的梯度，在 RGB 通道上取绝对值的最大值

用途：可以用于可视化，可以用于无监督分割，可以用于发现数据偏差。

7、请补充完整下面显著性可视化的代码

(理解可视化，体验即可)

```
1 import torch
2 import torchvision
3 import torchvision.transforms as T
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import requests
7 from PIL import Image
8
9 def download(url, fname):
10     response = requests.get(url)
11     with open(fname, "wb") as f:
12         f.write(response.content)
13
14 def preprocess(image, size=224):
15     transform = T.Compose([
16         T.Resize((size, size)),
17         T.ToTensor(),
18         T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
19         T.Lambda(lambda x: x[None]),
20     ])
21     return transform(image)
22
23 def deprocess(image):
24     transform = T.Compose([
25         T.Lambda(lambda x: x[0]),
26         T.Normalize(mean=[0, 0, 0], std=[4.3668, 4.4643, 4.4444]),
```

```

26         T.Normalize(mean=[-0.485, -0.456, -0.406], std=[1, 1, 1]),
27         T.ToPILImage(),
28     ])
29     return transform(image)
30
31 def show_img(PIL_IMG):
32     plt.imshow(np.asarray(PIL_IMG))
33
34 if __name__ == '__main__':
35     model = torchvision.models.vgg19(pretrained=True)
36     for param in model.parameters():
37         param.requires_grad = False
38
39     download("https://bking.cdn.bcebos.com/pic/3bf33a87e950352ac65cae81db13ecf2
b21192131da3?x-bce-
process=image/format,f_auto/quality,Q_70/resize,m_lfit,limit_1,w_536","input
.jpg")
40     img = Image.open('input.jpg') # 这里可以替换为自己的图片
41
42     x = preprocess(img) # x.shape: 1, 3, 224, 224
43     model.eval()
44     x.requires_grad_()
45     output = model(x) # 1, 1000
46     print(output.shape)
47     score_max_index = output.argmax()
48     score_max = output[0, score_max_index]
49
50     #####
51     # 需要实现课件上显著性可视化部分，具体的步骤为：
52     # a) 通过反向传播获取梯度
53     # b) 在RGB通道上取梯度绝对值的最大值
54     # 请在下面补充代码：
55     score_max.backward()
56     saliency, _ = torch.max(X.grad.data.abs(), dim=1)
57
58
59
60     #####
61
62
63     plt.imshow(saliency[0], cmap=plt.cm.hot)
64     plt.axis('off')
65     plt.savefig("output.png")

```

